Assignment 8 FAQ

Advanced Embedded Software Development

with **Dan Walkes**



University of Colorado **Boulder**

Learning objectives:

Learn tips and strategies to use when troubleshooting Assignment 8 Implementation

Assignment 8 Questions

- Why does the read function count value equal 131072 (0x20000)?
  - Probably the default buffer size of the underlying system call

```
echo "hi" > /dev/aesdchar
strace -o /tmp/strace.txt cat /dev/aesdchar
```

```
openat(AT_FDCWD, "/dev/aesdchar", O_RDONLY) = 3
fstat(3, {st_mode=S_IFCHR|0664, st_rdev=makedev(240, 0), ...}) = 0
fadvise64(3, 0, 0, POSIX_FADV_SEQUENTIAL) = 0
mmap(NULL, 139264, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fc6dbd37000
read(3, "hi\n", 131072)                 = 3
write(1, "hi\n", 3)                     = 3
read(3, "", 131072)                     = 0
munmap(0x7fc6dbd37000, 139264)          = 0
close(3)                                = 0
close(1)                                = 0
close(2)                                = 0
exit_group(0)                           = ?
```

Assignment 8 Questions

- My aesd_read function is continuously called in a loop. Why?
  - You probably didn't return 0 to say there's no remaining data to read

```
echo "hi" > /dev/aesdchar
strace -o /tmp/strace.txt cat /dev/aesdchar
```

```
openat(AT_FDCWD, "/dev/aesdchar", O_RDONLY) = 3
fstat(3, {st_mode=S_IFCHR|0664, st_rdev=makedev(240, 0), ...}) = 0
fadvise64(3, 0, 0, POSIX_FADV_SEQUENTIAL) = 0
mmap(NULL, 139264, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fc6dbd37000
read(3, "hi\n", 131072)                 = 3
write(1, "hi\n", 3)                     = 3
read(3, "", 131072)                     = 0
munmap(0x7fc6dbd37000, 139264)          = 0
close(3)                                = 0
close(1)                                = 0
close(2)                                = 0
exit_group(0)                           = ?
```

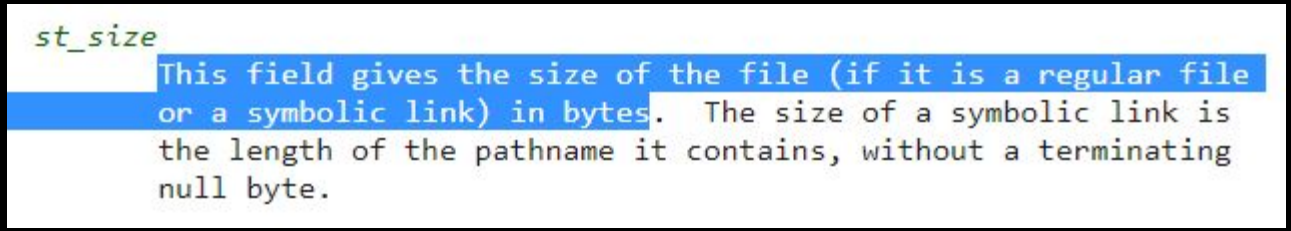
read return values

```
ssize_t read(struct file *filp, char __user *buff,
    size_t count, loff_t *offp);
```

- If return == count
  - requested number of bytes were transferred
- If 0 < return < count
  - only a portion has been returned
- 0 - end of file (no data read into buff)
- Negative - error (-ERESTARTSYS, -EINTR, -EFAULT)

Assignment 8 Questions

- Why doesn't fseek or stat work to find the size of /dev/ in aesdsocket?
  - fseek doesn't work because we aren't implement necessary write() seek support in our simple driver for assignment 8.
  - stat doesn't work because /dev/aesdchar is not a file



https://linux.die.net/man/2/stat

Assignment 8 Questions

- Why doesn't fseek or stat work to find the size of /dev/ in aesdsocket?
  - fseek doesn't work because we aren't implement necessary write() seek support in our simple driver for assignment 8.
  - stat doesn't work because /dev/aesdchar is not a file
- How do I get the size of the /dev/aesdchar device from aesdsocket?
  - You can't with our simple implementation for assignment 8
  - Easiest option is to remove logic which requires this.
    - read and write to file descriptor in a loop until EOF

https://linux.die.net/man/2/stat

Assignment 8 Questions

- Why isn't locking working? My system hangs...

```
//mutex_lock_interruptible(&dev->lock);

if( dev->Ini == dev->Outi || dev->read_check == 1 )
{
        dev->read_check = 0;
        PDEBUG("        FLAG clear %ld", retval);
        return retval;
}

//mutex_unlock(&dev->lock);
PDEBUG("retval=%ld",retval);
return retval;
```

- Handle locking in error paths
- Consider using goto

https://linux.die.net/man/2/stat

Ubuntu Crash reports

```
DEBUG = y

# Add your debugging flag (or not) to CFLAGS
ifeq ($(DEBUG),y)
  DEBFLAGS = -O -g -DSCULL_DEBUG # "-O" is needed to expand inlines
else
```

```
sudo cat /var/crash/201911042313/dmesg.201911042313
```

```
[  772.502161] aesdchar: write offset f_pos 6 is within existing comma
[  772.502196] BUG: unable to handle kernel paging request at ffff8c1a
[  772.502199] #PF error: [PROT] [WRITE]
[  772.502201] PGD 16ec01067 P4D 16ec01067 PUD 16ec05067 PMD 210a25063
[  772.502204] Oops: 0003 [#1] SMP PTI
[  772.502207] CPU: 4 PID: 4139 Comm: dd Kdump: loaded Tainted: G
[  772.502208] Hardware name: innotek GmbH VirtualBox/VirtualBox, BIOS
[  772.502212] RIP: 0010:__memcpy+0x12/0x20
[  772.502214] Code: 9b e8 72 15 6d ff 0f 31 48 c1 e2 20 48 09 d0 48 3
[  772.502215] RSP: 0018:ffff9b07062f3e20 EFLAGS: 00010202
[  772.502217] RAX: ffff8c1acf429ec8 RBX: ffffffffc0555440 RCX: 1fffff
[  772.502218] RDX: 0000000000000007 RSI: ffff8c1acf485780 RDI: ffff8c
[  772.502219] RBP: ffff9b07062f3e88 R08: ffff8c1ad7a28040 R09: ffff8c
[  772.502220] R10: 0000000000000001 R11: 0000000000000001 R12: 000000
[  772.502221] R13: ffff8c1acf429ec8 R14: ffffffffc0555550 R15: ffffff
[  772.502222] FS:  00007fdd1de4b540(0000) GS:ffff8c1ad7a00000(0000) k
[  772.502224] CS:  0010 DS: 0000 ES: 0000 CR0: 0000000080050033
[  772.502225] CR2: ffff8c1acf486000 CR3: 0000000210786000 CR4: 000000
[  772.502229] DR0: 0000000000000000 DR1: 0000000000000000 DR2: 0000000000000000
[  772.502230] DR3: 0000000000000000 DR6: 00000000fffe0ff0 DR7: 0000000000000400
[  772.502231] Call Trace:
[  772.502236]  ? aesd_write+0x1fc/0x3e6 [aesdchar]
[  772.502240]  __vfs_write+0x1b/0x40
[  772.502242]  vfs_write+0xb1/0x1a0
[  772.502244]  ksys_write+0x5c/0xe0
[  772.502247]  __x64_sys_write+0x1a/0x20
[  772.502249]  do_syscall_64+0x5a/0x120
[  772.502252]  entry_SYSCALL_64_after_hwframe+0x44/0xa9
[  772.502253] RIP: 0033:0x7fdd1d95d154
```

```
objdump -S aesdchar.ko | less
```

```
0000000000000669 <aesd_write>:
{
 669:   e8 00 00 00 00          callq  66e <aesd_write+0x5>
                        memcpy(newbuffer,working_entry->buffptr,buffer_offs);
 84b:   48 8b 45 c8             mov    -0x38(%rbp),%rax
 84f:   48 8b b3 30 01 00 00    mov    0x130(%rbx),%rsi
 856:   48 89 45 a8             mov    %rax,-0x58(%rbp)
 85a:   48 89 c2                mov    %rax,%rdx
 85d:   4c 89 ef                mov    %r13,%rdi
 860:   e8 00 00 00 00          callq  865 <aesd_write+0x1fc>
                        memcpy(&newbuffer[buffer_offs],working_entry->buffptr,working_entry->size);
 865:   48 8b 45 a8             mov    -0x58(%rbp),%rax
 869:   49 8d 7c 05 00          lea    0x0(%r13,%rax,1),%rdi
 86e:   48 8b b3 30 01 00 00    mov    0x130(%rbx),%rsi
 875:   48 8b 93 38 01 00 00    mov    0x138(%rbx),%rdx
```

0x669 = address of aesd_write start

0x1fc is the offset within aesd_write where the oops occurred

0x669 + 0x1fc = 0x865 - location where the oops occurred

```
aesdchar: write offset f_pos 6 is within existing commands at entry offset 1 with buffer offset -1
BUG: unable to handle kernel paging request at ffff8c1acf486000
```

https://askubuntu.com/questions/346953/how-to-read-and-use-crash-reports

Assignment 8 Questions

- “I’m not seeing printk messages from the driver when I use echo”
- “When I use drivertest.sh it works”

```
echo "check" > sudo /dev/aesdchar
```

- What does this do?
  - Writes “check /dev/aesdchar to a new file named sudo
  - Use sudo echo “check” > /dev/aesdchar
  - Better yet: fix permissions so you don’t need sudo

```
echo "check" > sudo /dev/aesdchar

ecen5013@ecen5013-VirtualBox:~/assignments-complete-private/aesd-char-driver$ cat sudo
check /dev/aesdchar

execve("/bin/echo", ["echo", "check", "/dev/aesdchar"], 0x7ffd78dbc5d0 /* 22 vars */) = 0
write(1, "check /dev/aesdchar\n", 20)   = 20
```

Permissions

- When running on your host, add your user to the “staff” group to run drivertest and sockettest scripts without root permissions
  - sudo usermod -a -G staff <yourusername>
  - logout/back in

```
#!/bin/sh
module=aesdchar
device=aesdchar
mode="664"
cd `dirname $0`
set -e
# Group: since distributions do it differently, look for wheel or use staff
if grep -q '^staff:' /etc/group; then
    group="staff"
else
    group="wheel"
fi

insmod ./$module.ko $* || exit 1
major=$(awk "\$2==\"$module\" {print \$1}" /proc/devices)
rm -f /dev/${device}
mknod /dev/${device} c $major 0
chgrp $group /dev/${device}
chmod $mode  /dev/${device}
"aesdchar_load" 19L, 435C written
```

Helper Scripts

- Use helper scripts when you find yourself performing the same commands repeatedly

```
ecen5013@ecen5013-VirtualBox:~/assignments-complete-private$ cat socketstart-driver.sh
sudo ./aesd-char-driver/aesdchar_unload
sudo ./aesd-char-driver/aesdchar_load
strace -o /tmp/strace-aesdsocket.txt -f ./aesdsocket
```

Assignment 8 Suggestions

- Implement in stages to get ./drivertest.sh working
  - First implement a simple version that assumes 1 write = 1 command
  - Verify this works for simple cases with echo and cat on the command line.
  - Save *and push* this commit
  - Add locking
  - Repeat
  - Add support for partial command writes
  - Repeat
- If your driver crashes/hangs
  - Look *very closely* at what you changed since your last working commit